

1. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

```
import numpy as np

X = np.array([[2, 9], [1, 5], [3, 6]], dtype=float)
y = np.array([[92], [86], [89]], dtype=float)

# Normalize input data
X = X / np.amax(X, axis=0)
y = y / 100

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid function
def derivatives_sigmoid(x):
    return x * (1 - x)

# Variable initialization
epoch = 5000                                # Training iterations
lr = 0.1                                    # Learning rate
inputlayer_neurons = 2                      # Number of input features
hiddenlayer_neurons = 3                    # Number of hidden layer neurons
output_neurons = 1                          # Number of output neurons

# Weight and bias initialization
wh = np.random.uniform(size=(inputlayer_neurons,
                               hiddenlayer_neurons))
bh = np.random.uniform(size=(1, hiddenlayer_neurons))

wout = np.random.uniform(size=(hiddenlayer_neurons, output_neurons))
bout = np.random.uniform(size=(1, output_neurons))

# Training the network
for i in range(epoch):

    # Forward Propagation
    hinp1 = np.dot(X, wh)
    hinp = hinp1 + bh
    hlayer_act = sigmoid(hinp)

    outinp1 = np.dot(hlayer_act, wout)
    outinp = outinp1 + bout
    output = sigmoid(outinp)

    # Backpropagation
    EO = y - output
```

```
outgrad = derivatives_sigmoid(output)
d_output = E0 * outgrad

EH = d_output.dot(wout.T)

# Hidden layer contribution to error
hiddengrad = derivatives_sigmoid(hlayer_act)
d_hiddenlayer = EH * hiddengrad

# Updating weights
wout += hlayer_act.T.dot(d_output) * lr
wh += X.T.dot(d_hiddenlayer) * lr

# Display results
print("Input: \n", X)
print("Actual Output: \n", y)
print("Predicted Output: \n", output)
```